Artificial Intelligence

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Clase # 11 - PAO 2024 – II

Agenda

1. El algoritmo GD con backprop en MLP
2. Introducción a las redes neuronales convolucionales – CNN

Aprendizaje: Entrenamiento con GD

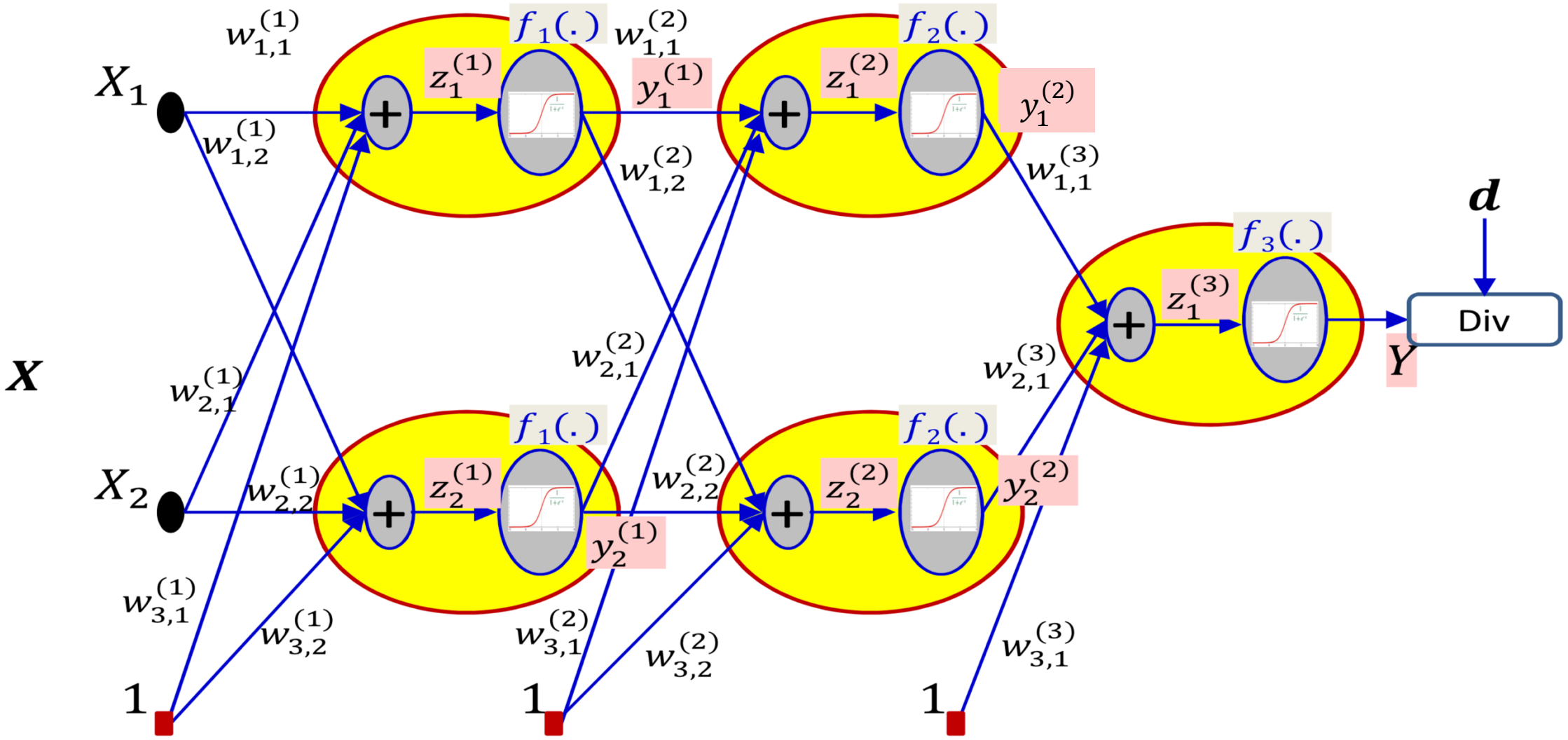
Algoritmo Gradiente Descendiente:

- Inicializar todos los pesos $\left\{W_{i,j}^{(k)}\right\}$
- Realizar:
  - Para todas las i, j, k, inicializar $\frac{dError}{dW_{i,j}^{(k)}} = 0$
  - Para todas los ejemplos $t = 1{:}\,T$
    - Para cada capa k y para todas las unidades i, j:
      - Calcular: $\frac{dDiv(Y_t, d_t)}{dW_{i,j}^{(k)}}$
      - Calcular: $\frac{dError}{dW_{i,j}^{(k)}} \mathrel{+}= \frac{dDiv(Y_t, d_t)}{dW_{i,j}^{(k)}}$
  - Para cada capa k y para todas las unidades i, j:

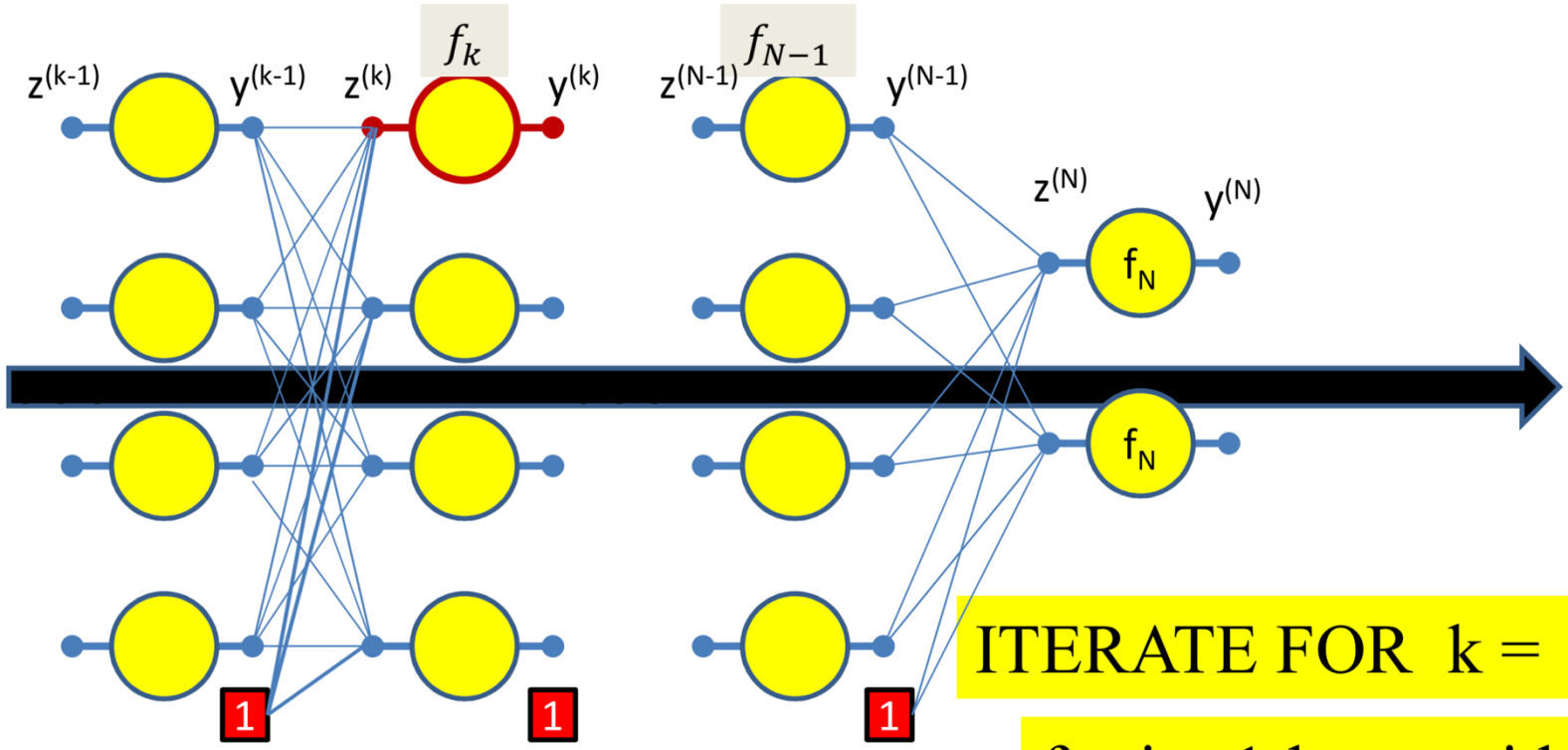
$$W_{i,j}^{(k)} = W_{i,j}^{(k)} - \propto \frac{dError}{dW_{i,j}^{(k)}}$$

  - Hasta que el $Error$ converja

Aprendizaje:



Aprendizaje: Forward



ITERATE FOR k = 1:N

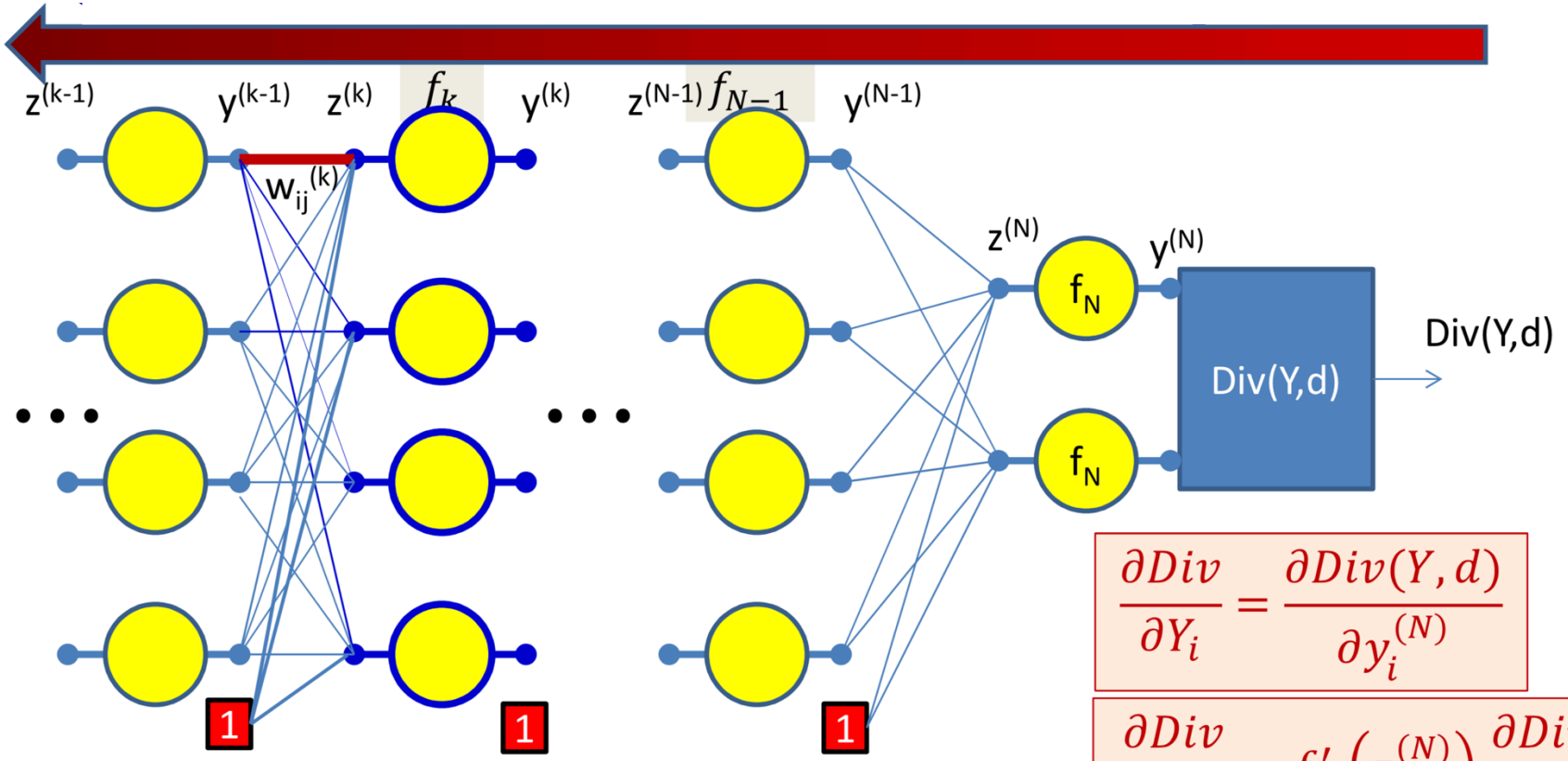
for j = 1:layer-width

$$y_i^{(0)} = x_i$$

$$z_j^{(k)} = \sum_i w_{ij}^{(k)} y_i^{(k-1)}$$

$$y_j^{(k)} = f_k\left(z_j^{(k)}\right)$$

Aprendizaje: Backward



$$\frac{\partial Div}{\partial w_{ij}^{(k)}} = \frac{\partial z_j^{(k)}}{\partial w_{ij}^{(k)}} \frac{\partial Div}{\partial z_j^{(k)}} = y_i^{(k-1)} \frac{\partial Div}{\partial z_j^{(k)}}$$

$$\frac{\partial Div}{\partial Y_i} = \frac{\partial Div(Y,d)}{\partial y_i^{(N)}}$$

$$\frac{\partial Div}{\partial z_i^{(N)}} = f_N'\left(z_i^{(N)}\right) \frac{\partial Div}{\partial y_i^{(N)}}$$

$$\frac{\partial Div}{\partial y_i^{(k-1)}} = \sum_j w_{ij}^{(k)} \frac{\partial Div}{\partial z_j^{(k)}}$$

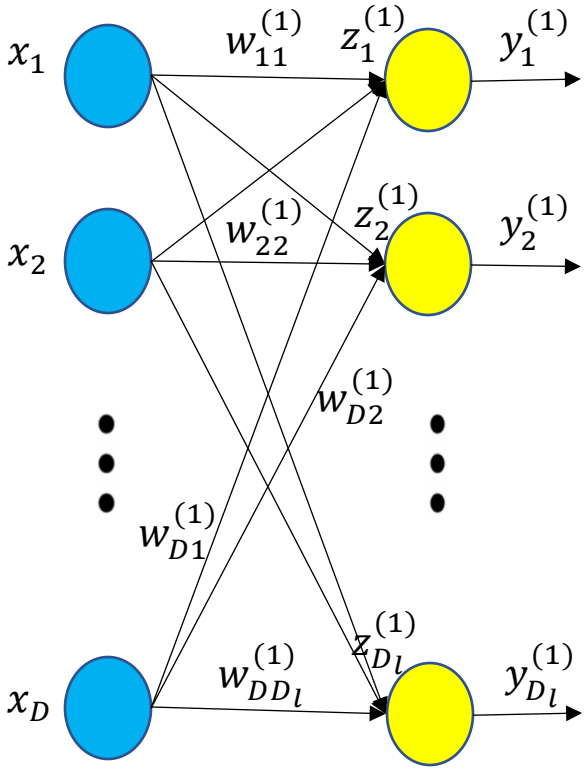
Aprendizaje: Entrenamiento con BP

- Inicializar todos los pesos $\{W^{(1)}, W^{(2)}, \ldots, W^{(K)}\}$
- Realizar:
  - Inicializar $Error = 0$, Para todas las i, j, k, inicializar $\frac{dError}{dW_{i,j}^{(k)}} = 0$
  - Para todas los $t = 1{:}\,T$
    - Para cada capa k y para todas las unidades i, j:
      - Paso "Forward":
        - Calcular la salida Y_t
        - Calcular $Error \mathrel{+}= Div(Y_t, d_t)$
      - Paso "Backward":
        - Calcular: $\frac{dDiv(Y_t, d_t)}{dW_{i,j}^{(k)}}$
        - Calcular: $\frac{dError}{dW_{i,j}^{(k)}} \mathrel{+}= \frac{dDiv(Y_t, d_t)}{dW_{i,j}^{(k)}}$
  - Para todas las i, j, k actualizar:

$$W_{i,j}^{(k)} = W_{i,j}^{(k)} - \frac{\propto}{T} \frac{dError}{dW_{i,j}^{(k)}}$$

  - Hasta que el $Error$ converja

Aprendizaje: En Deep Learning – formulación vectorial



$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_D \end{bmatrix} \qquad Z_k = \begin{bmatrix} z_1^{(k)} \\ z_2^{(k)} \\ \vdots \\ z_{D_k}^{(k)} \end{bmatrix} \qquad y_k = \begin{bmatrix} y_1^{(k)} \\ y_2^{(k)} \\ \vdots \\ y_{D_k}^{(k)} \end{bmatrix}$$

$$W = \begin{bmatrix} w_{11}^{(k)} & w_{21}^{(k)} & \cdots & w_{D_k 1}^{(k)} \\ w_{12}^{(k)} & w_{22}^{(k)} & \cdots & w_{D_k 2}^{(k)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{1D_{l+1}}^{(k)} & w_{2D_{l+1}}^{(k)} & \cdots & w_{D_{l+1}D_{l+1}}^{(k)} \end{bmatrix} \qquad b_k = \begin{bmatrix} b_1^{(k)} \\ b_2^{(k)} \\ \vdots \\ b_{D_{l+1}}^{(k)} \end{bmatrix}$$

- El cálculo en forma vectorial de una capa se expresa:

$$Z_k = W_k y_{k-1} + b_k$$
$$y_k = f_k(Z_k)$$

Aprendizaje: En Deep Learning – formulación vectorial

- La derivada de una función vectorial c.r.a un vector de entrada se llama "Jacobian"
- Es una matriz de derivadas parciales, que tiene la forma:

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_M \end{bmatrix} = f\left(\begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_D \end{bmatrix}\right)$$

En notación vectorial

$$\mathbf{y} = f(\mathbf{z})$$

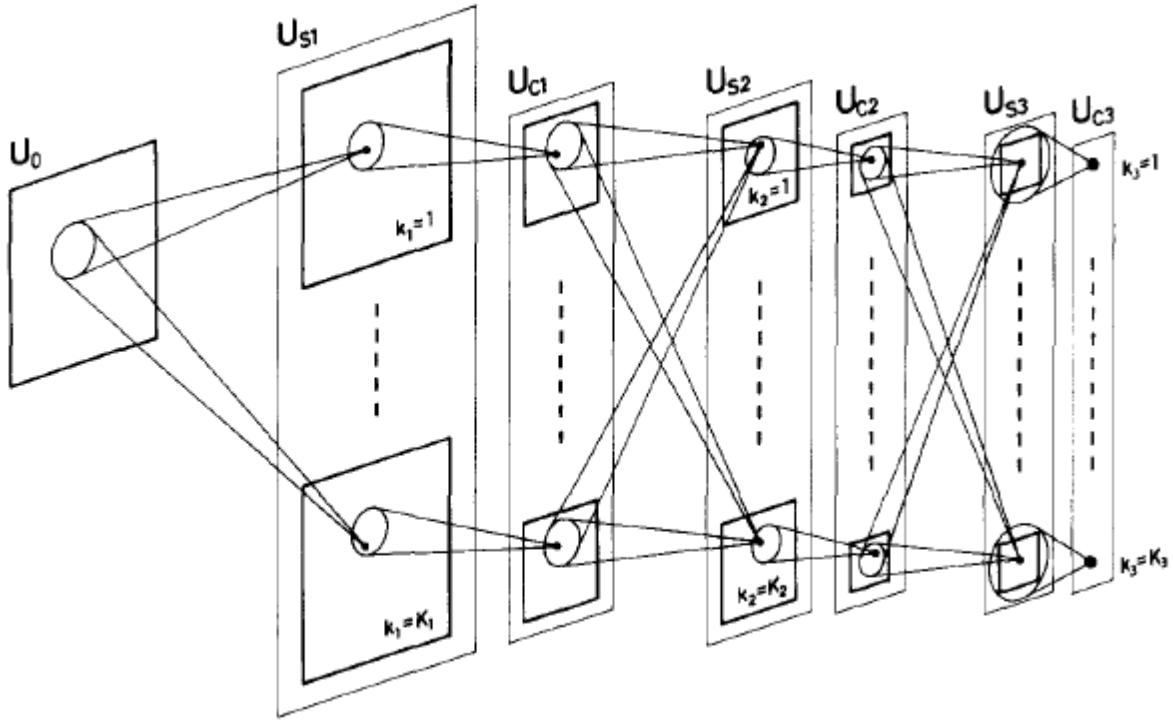
$$J_{\mathbf{y}}(\mathbf{z}) = \begin{bmatrix} \frac{\partial y_1}{\partial z_1} & \frac{\partial y_1}{\partial z_2} & \cdots & \frac{\partial y_1}{\partial z_D} \\ \frac{\partial y_2}{\partial z_1} & \frac{\partial y_2}{\partial z_2} & \cdots & \frac{\partial y_2}{\partial z_D} \\ \cdots & \cdots & \ddots & \cdots \\ \frac{\partial y_M}{\partial z_1} & \frac{\partial y_M}{\partial z_2} & \cdots & \frac{\partial y_M}{\partial z_D} \end{bmatrix}$$

$$\Delta y = J_y(z)\Delta z$$

Redes neuronales convolucionales

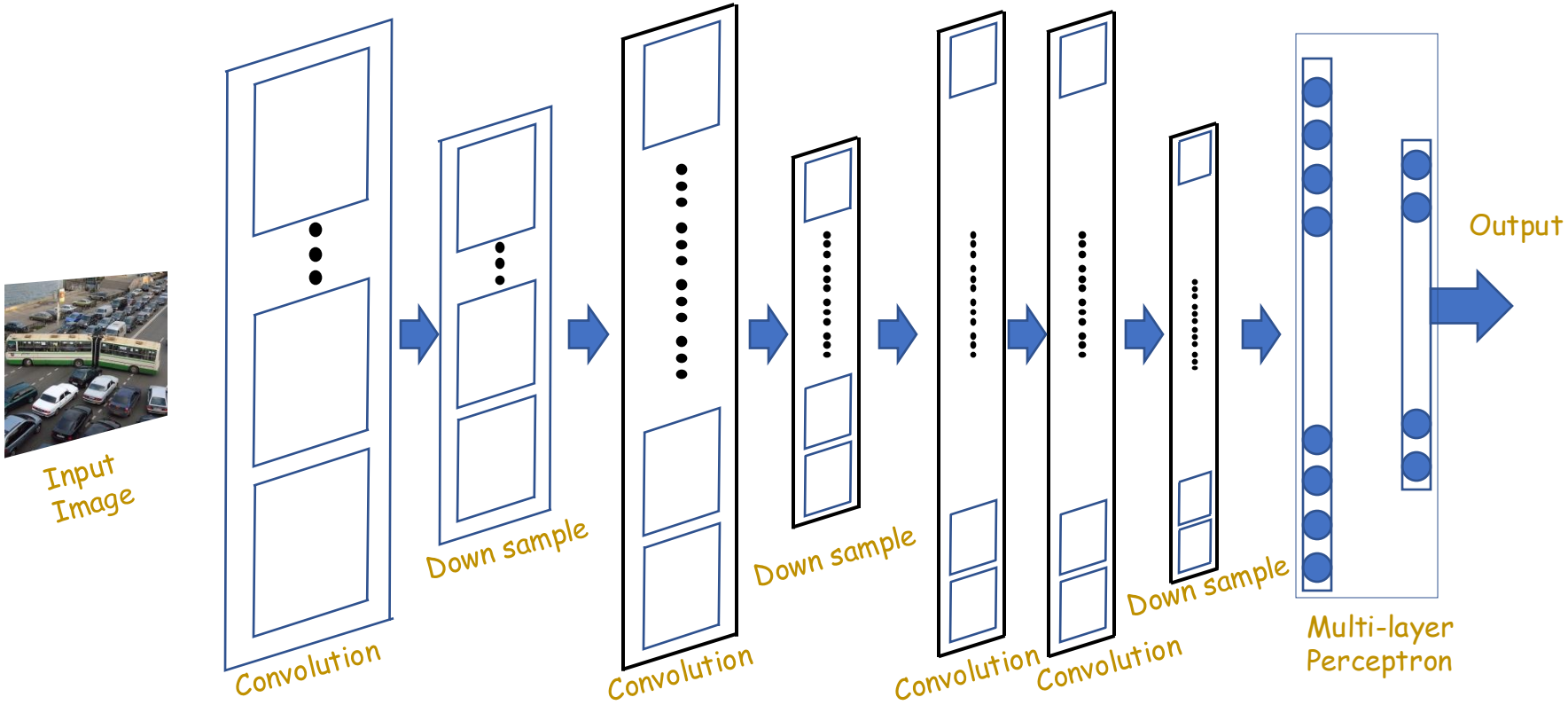
- Las redes CNN son una categoría de redes neuronales que han demostrado ser muy eficaces en áreas como el reconocimiento y la clasificación de patrones
- ConvNets ha tenido éxito en la identificación de rostros, objetos y señales de tráfico, además de potenciar la visión utilizada en autos, que se conducen solos,…

Neocognitron

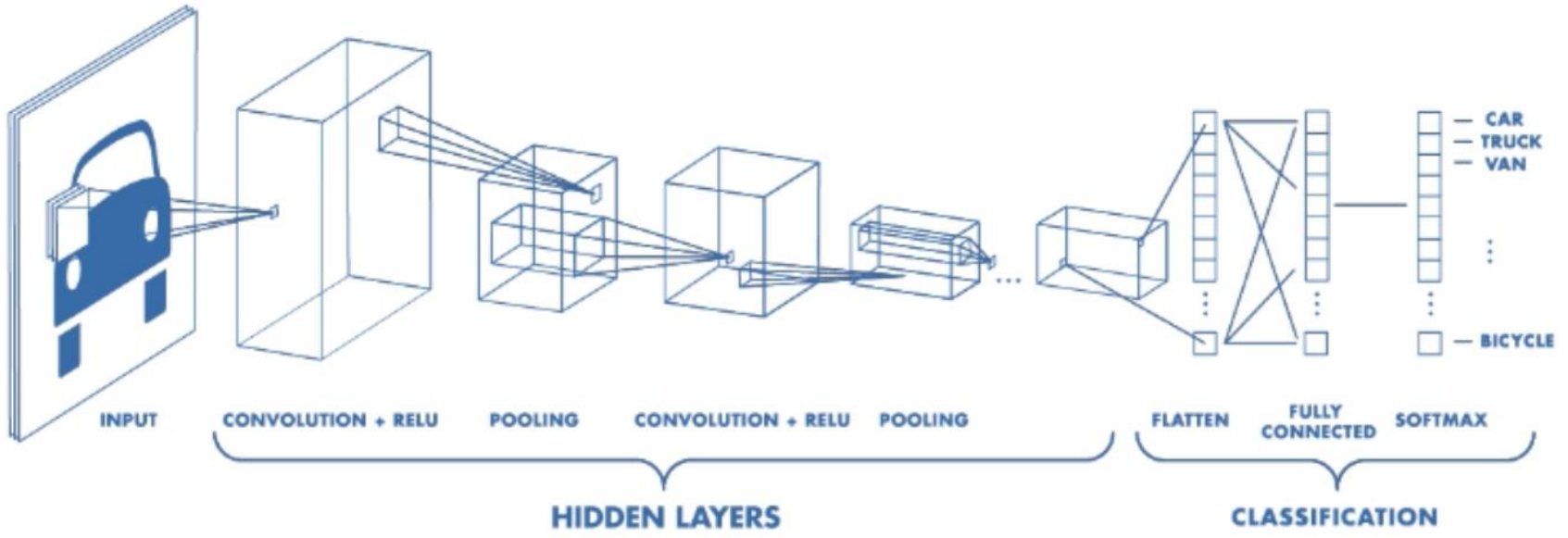


- Las células S detectan patrones específicos en la capa anterior (capa C o retina)
- Las células C "refinan" la respuesta de los planos correspondientes de las capas S

Redes neuronales convolucionales



Redes neuronales convolucionales



Architecture of a CNN. — Source: https://www.mathworks.com/videos/introduction-to-deep-learning-what-are-convolutional-neural-networks--1489512765771.html

Convolution

Imagen de 5x5

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Filtro de 3x3

1	0	1
0	1	0
1	0	1

bias

0

$$z(i,j) = \sum_{k=1}^{3}\sum_{l=1}^{3} f(k,l)I(i+l,j+k) + b$$

- Barrido de la imagen con un “filtro”
  - Notar: El filtro es un perceptron, con pesos y un bias

Convolution



- Barrido de la imagen con un “filtro”
  - El filtro puede avanzar en más de 1 píxel a la vez.
  - Por ejemplo: con un paso o ”stride" de dos píxeles por vez

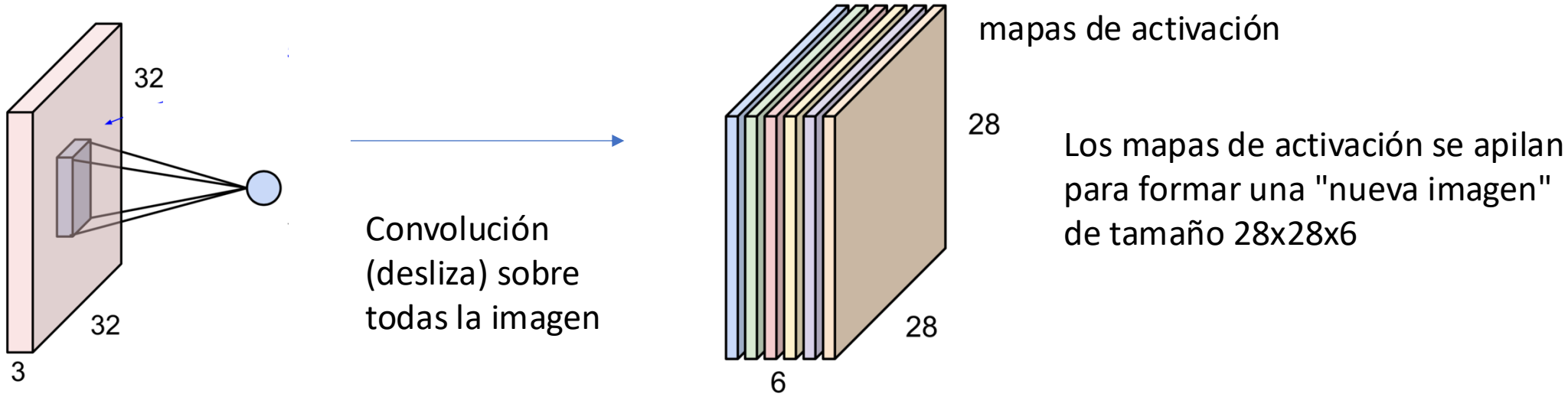
Redes neuronales convolucionales

Capas utilizadas en la construcción de CNNs

- Las CNNs se componen de una secuencia de capas, y cada capa transforma un volumen de activaciones a otra a través de una función diferenciable
- Hay tres tipos principales de capas:
  - Capa convolucional,
  - Capa de pooling, y
  - Capa totalmente conectada (tal como en las RN).

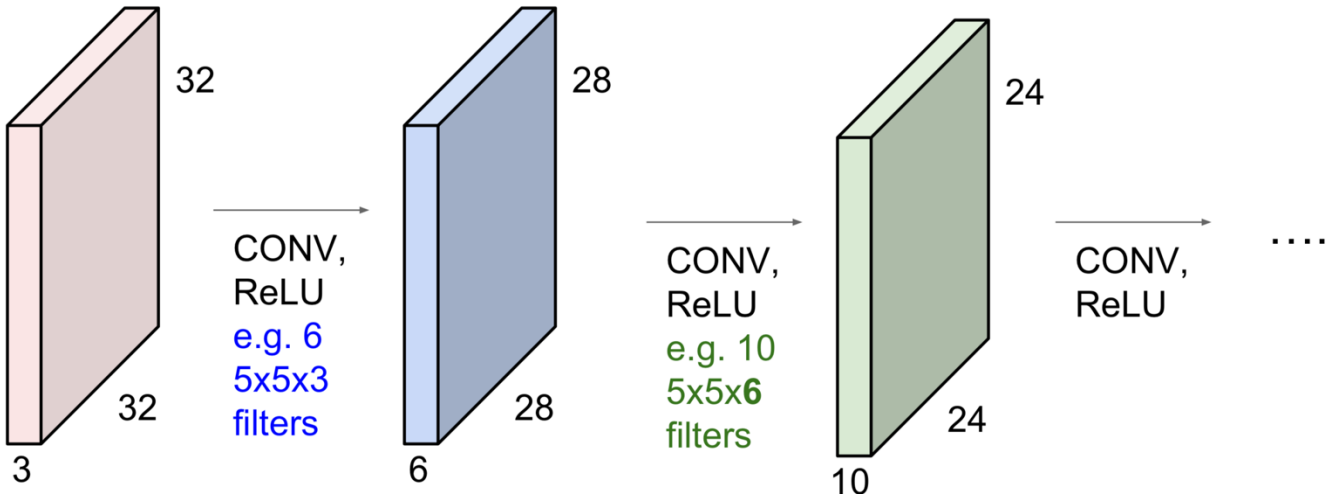
Redes neuronales convolucionales

- En las CNN las capas están organizadas en 3 dimensiones: ancho, alto y profundidad
- Las neuronas en una capa no se conectan a todas las neuronas de la siguiente capa, sino sólo a una pequeña región de la misma



Redes neuronales convolucionales

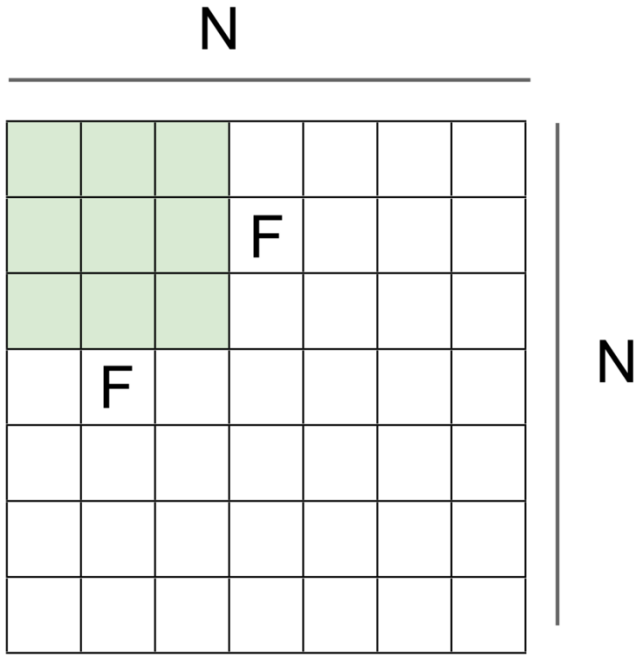
- En las CNN las capas están organizadas en 3 dimensiones: ancho, alto y profundidad
- Las neuronas en una capa no se conectan a todas las neuronas de la siguiente capa, sino sólo a una pequeña región de la misma



Redes neuronales convolucionales

- Stride (o paso) es el tamaño del paso que el filtro de convolución utiliza cada vez que se mueve
- Por ejemplo un paso de tamaño 1, significa que el filtro se desliza píxel por píxel.
- Al aumentar el tamaño del paso, el filtro se desliza sobre la entrada con un intervalo mayor y por lo tanto tiene menos superposición entre las celdas.

Redes neuronales convolucionales



Tamaño de la salida:

(N - F) / stride + 1

Ej: N = 7, F = 3:

stride 1 => (7 - 3)/1 + 1 = 5

stride 2 => (7 - 3)/2 + 1 = 3

stride 3 => (7 - 3)/3 + 1 = 2.33